




SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE:** CSA3711

CODE OF CONDUCT

I Rakcha A(192311421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rakcha A

Course Outcome Covered

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

Assessment Tool Weightage: 20%

Note: Where the question paper does not supply the test design, code or data, a representative case is assumed and stated explicitly in the answer; the error-analysis method remains the same.

Question 1

A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

Answer:

1.1 Given Test Design (Assumed)

The question does not list the original test cases, so a typical flawed design for the "Quantity" field of the cart is assumed. Business rule: a customer may order 1 to 10 units of an item per order.

Class (as designed)	Values placed in the class	Test value chosen
Class A: "Valid quantities"	0, 1, 5, 10, 11	5
Class B: "Invalid quantities"	-3, abc, 20	-3

1.2 Errors Identified

No.	Error	Explanation
E1	Valid and invalid values mixed in Class A	0 and 11 are invalid but are placed with valid values 1, 5, 10; the class is not homogeneous
E2	Invalid class is too broad	-3, "abc" and 20 fail for different reasons (negative, non-numeric, above maximum) and give different error messages
E3	Missing invalid classes	Decimal (2.5), blank, special characters and very large numbers are not partitioned
E4	One representative per class	The tester picks 5 and -3 only, which proves nothing about the other values in the class

No.	Error	Explanation
E5	No boundary values (BVA missing)	0, 1, 10 and 11 are never tested as boundaries; defects such as "<" written instead of "<=" cannot be found
E6	Boundary values treated as ordinary members	1 and 10 are in the valid class but are never selected as test data

1.3 Root Cause Analysis

- **Misunderstanding of the principle:** equivalence partitioning requires every member of a class to be treated the same way by the system. Mixing valid and invalid values breaks this assumption.
- **Partitioning by value instead of by behavior:** the tester grouped values by appearance (numbers versus others) and not by the response of the system (accept, reject with message X, reject with message Y).
- **Requirement not analysed:** the 1 to 10 rule was not converted into partitions, so no limits were identified.
- **EP and BVA treated as the same technique:** BVA is a refinement of EP that tests the edges of each ordered class; omitting it is the commonest cause of off-by-one defects.
- **No review or traceability:** the design was not checked against requirements, so incorrect classes were not detected.

1.4 Corrected Partitions

Class	Type	Range / description	Representative value	Expected result
V1	Valid	1 <= qty <= 10	5	Added to cart
I1	Invalid	qty < 1 (zero and negatives)	-3	Error: quantity must be at least 1
I2	Invalid	qty > 10	20	Error: maximum 10 per order
I3	Invalid	Decimal number	2.5	Error: whole numbers only
I4	Invalid	Non-numeric / special characters	abc, @#	Error: enter a number
I5	Invalid	Blank / null	(empty)	Error: quantity required
I6	Invalid	Extremely large	99999999999	Error; no crash or overflow

1.5 Corrected Test Cases with Boundary Coverage

TC	Technique	Input	Expected result
1	EP (V1)	5	Accepted
2	BVA lower - 1	0	Rejected: minimum is 1
3	BVA lower	1	Accepted
4	BVA lower + 1	2	Accepted
5	BVA upper - 1	9	Accepted
6	BVA upper	10	Accepted
7	BVA upper + 1	11	Rejected: maximum is 10
8	EP (I1)	-3	Rejected
9	EP (I2)	20	Rejected
10	EP (I3)	2.5	Rejected: whole numbers only
11	EP (I4)	abc	Rejected: not a number
12	EP (I5)	blank	Rejected: required
13	EP (I6)	99999999999	Rejected without crash

1.6 Justification

- Homogeneous classes mean one test per class is a valid representative, so every failure type (negative, decimal, text, blank, too big) is exercised and each error message is verified.
- The six boundary tests (0, 1, 2, 9, 10, 11) detect off-by-one defects (<, <=, >, >=), which are the most frequent input-validation defects.
- The original design executed 2 tests and exercised 2 behaviors; the corrected design uses 13 tests and exercises all 7 classes and all 4 boundary edges, so defect detection improves without redundant testing.
- Every test is traceable to a requirement or class, which makes the design reviewable and repeatable.

Question 2

A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.

Answer:

2.1 Evaluation of the Approach

Random testing means entering arbitrary values and operations without a model of the requirements. Assume the bank rules: minimum transfer Rs. 1; maximum Rs. 50,000 per transaction; daily limit Rs. 2,00,000; minimum balance Rs. 1,000; transfers are allowed only for active accounts. The engineer ran a few hundred random transfers and reported "high coverage".

2.2 Errors in Strategy and Coverage Assumptions

No.	Error	Why it is wrong
E1	No test design technique used	Tests are not derived from requirements, so there is no assurance that each rule is tested
E2	"High coverage" claimed without a measure	Coverage must be defined against something (requirements, partitions, boundaries, code); the number of tests is not coverage
E3	Assumption that many tests give wide coverage	Random inputs cluster in the middle of the range and repeat the same behavior
E4	Boundaries ignored	The chance of picking exactly 49,999, 50,000 or 50,001 from 1 to 10,00,000 is about 3 in 1,000,000 for each test
E5	Combinations and states ignored	Daily limit, balance and account status interact; random single-field tests do not test combinations or sequences
E6	Not repeatable and not traceable	Random tests cannot be re-run after a fix, so regression is impossible
E7	No expected results (oracle)	Outputs cannot be judged correct or wrong without a predefined expected result

2.3 Root Cause Analysis of the Missed Defects

Missed defect	Root cause
Transfer of Rs. 50,001 accepted (limit check coded as ">")	The boundary value was never selected; BVA was not applied
Rs. 2,00,000 daily limit not enforced for the third transfer of the day	Random tests are independent; no cumulative or sequence-based scenario was designed
Balance falls below the Rs. 1,000 minimum	Interaction between balance and amount was not tested (no decision table)
Transfer from a frozen account succeeds	Account-status states were not modelled (no state transition testing)
Rs. 0 and negative amounts accepted	Invalid equivalence classes were never defined

Underlying causes: lack of test planning and training, pressure to report numbers, and no review of the test design against the requirements.

2.4 Suggested Black-Box Techniques

Technique	Application to the banking system	Example
Equivalence partitioning	Valid class 1 to 50,000; invalid < 1; invalid > 50,000; non-numeric	Rs. 25,000 valid; Rs. -10 invalid
Boundary value analysis	Test at and around every limit	0, 1, 2, 49,999, 50,000, 50,001; balance-after = 999 / 1,000 / 1,001
Decision table	Combinations of amount valid, balance sufficient, daily limit not exceeded, account active	16 rules; only rule "all Y" allows the transfer
State transition testing	Account states Active, Frozen, Closed; transfer sequences within a day	Frozen account: transfer rejected; unfrozen: accepted
Error guessing (supplementary)	Double-click on submit, session timeout, network drop	Debit happens exactly once

2.5 Sample Corrected Test Cases

TC	Technique	Input	Expected result
1	BVA	Amount 49,999 / 50,000	Accepted
2	BVA	Amount 50,001	Rejected: exceeds per-transaction limit
3	BVA	Amount 0 / 1	Rejected / accepted
4	Decision table	Amount valid, balance 40,000, amount 39,500 (remaining 500)	Rejected: below minimum balance

TC	Technique	Input	Expected result
5	Cumulative	Four transfers of 50,000 on one day	First four accepted (2,00,000); fifth rejected
6	State	Frozen account, valid transfer	Rejected: account frozen
7	State	Unfreeze then transfer	Accepted

2.6 Justification

- **Adequacy:** coverage is now measurable (every partition, boundary, rule and state transition is a coverage item), so "100% of partitions and boundaries" has a defined meaning.
- **Defect detection:** boundary and combination defects, the exact class that was missed, are targeted directly instead of by chance.
- **Reliability:** tests are repeatable, have expected results and can be used for regression after every fix, so results can be trusted and compared.
- **Efficiency:** a small designed set (about 40 tests) detects more serious defects than hundreds of random tests.

Question 3

A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Answer:

3.1 Code Under Test (Assumed)

Node	Code
1	total = 0; i = 0;
2	while (i < n) {
3	if (m[i] < 0)
4	invalid++;
5	else total += m[i];
6	i++; }
7	if (invalid > 0)
8	print "Errors";
9	else if (total > 100)
10	grade = "Pass";
11	else grade = "Fail";
12	return;

3.2 The Developer's Incomplete CFG

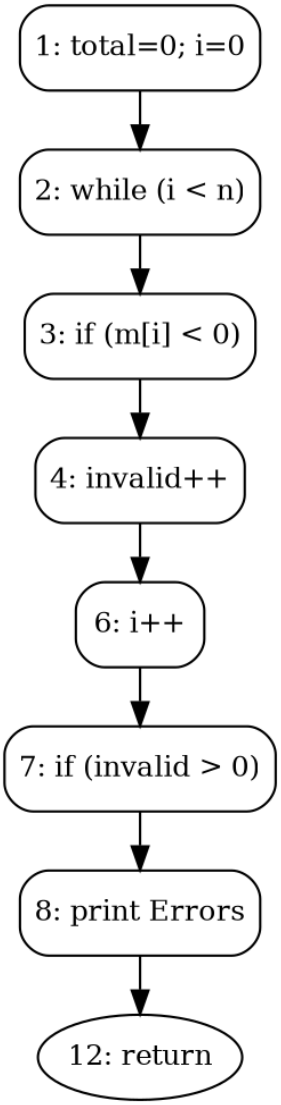


Figure 3.1: Incomplete CFG drawn by the developer

3.3 Errors Identified

No.	Error in the CFG / approach	Effect
E1	Loop-back edge from node 6 to node 2 is missing; the while loop is drawn as straight-line code	Zero, one and many iterations are never considered
E2	Loop exit edge (2 to 7) missing	The "loop not executed" path (n = 0) is lost
E3	Decision at node 3 drawn with one outgoing edge; else node 5 missing	The path through "total += m[i]" is never tested
E4	Decision at node 9 and nodes 10 and 11 missing	Pass and Fail outcomes are not tested
E5	Decision at node 7 has only the true edge	The false branch (no invalid marks) is not tested

No.	Error in the CFG / approach	Effect
E6	Only one path (1-2-3-4-6-7-8-12) identified; V(G) not calculated	Path coverage is 1 of 5 basis paths, i.e. 20%

3.4 Root Causes of Inadequate Coverage

- The CFG was drawn from the "happy path" and not from the code line by line, so decision nodes without visible else statements (the implicit else) were ignored.
- Misunderstanding that a decision node always has two outgoing edges and that a loop condition is a decision node with a back edge.
- No cyclomatic complexity calculation, so the developer did not know how many independent paths exist.
- No peer review of the graph against the code; test cases were derived from the wrong model, so the missing paths were never tested.

3.5 Corrected Control Flow Graph

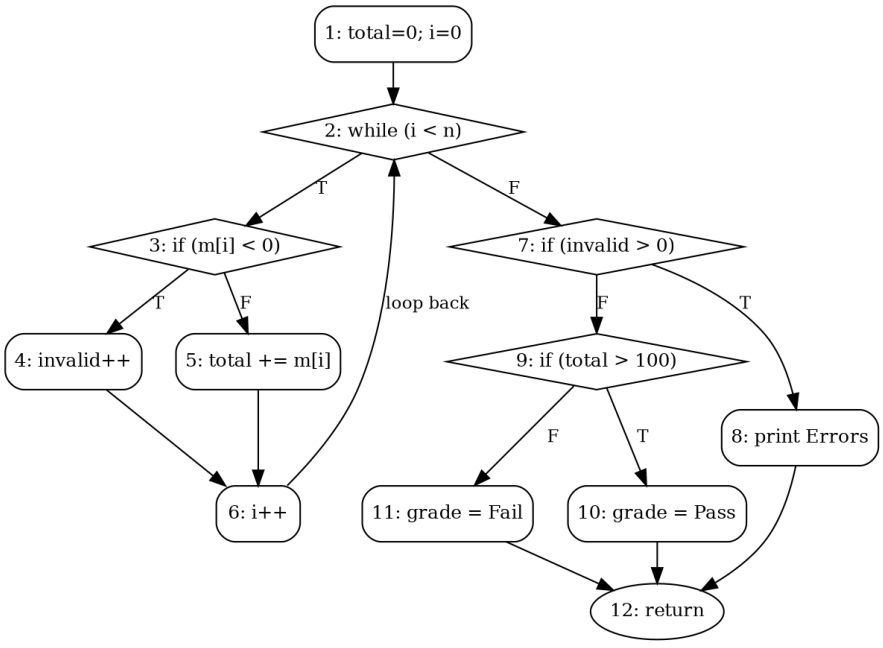


Figure 3.2: Corrected CFG

Nodes N = 12 and edges E = 15, so V(G) = E - N + 2 = 15 - 12 + 2 = **5**. Check: predicate nodes (2, 3, 7, 9) + 1 = 5.

3.6 Independent Paths and Test Cases

Path	Node sequence	Test data (n, m[])	Expected
P1	1-2-7-9-11-12	n = 0	Fail (loop skipped)

Path	Node sequence	Test data (n, m[])	Expected
P2	1-2-3-5-6-2-7-9-10-12	n = 1, m = [150]	Pass
P3	1-2-3-5-6-2-7-9-11-12	n = 1, m = [50]	Fail
P4	1-2-3-4-6-2-7-8-12	n = 1, m = [-5]	Errors printed
P5	1-2-3-4-6-2-3-5-6-2-7-8-12	n = 2, m = [-5, 60]	Errors printed (both branches of node 3 within one run)

3.7 Justification

- The corrected graph has every decision (2, 3, 7, 9) with both outcomes and the loop back edge, so the model matches the code and test cases derived from it are complete.
- Five basis paths give 100% branch coverage (8 of 8 decision outcomes) and test the loop with zero, one and several iterations.
- Defects such as a wrong comparison in node 9, a wrong update of total, or failure when n = 0 are now exposed, whereas the one-path design could not detect them.
- Because V(G) = 5 is known, the number of required tests is objective; coverage can be proven rather than assumed.

Question 4

A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

Answer:

4.1 Module Under Test (Assumed)

Specification: a loan is approved if the applicant is at least 18 years old **and** the monthly income is **at least** Rs. 25,000.

Line	Code (implementation)
1	approved = false;
2	if (age >= 18 && income > 25000) // defect: should be income >= 25000
3	approved = true;
4	return approved;

The team used one test, T1: age 30, income 50,000. Every statement (1 to 4) executes, so statement coverage is **100%**, yet the boundary defect is not detected.

4.2 Evaluation of the Adequacy of Testing

Coverage criterion	Achieved by T1	Verdict
Statement	100%	Met
Decision (branch)	50% (only the true outcome)	Not met
Condition	50% (each condition only true)	Not met
Path	50% (1 of 2 paths)	Not met
Boundary / MC/DC	0%	Not met

4.3 Gaps in Coverage Criteria

- The false outcome of the decision on line 2 is never executed, so the case where the loan is rejected is untested.
- Neither condition (age >= 18, income > 25000) is ever false, so errors in either condition (operators, constants) are invisible.
- No test at the boundary (age = 18, income = 25,000), where the defect lies.

- The combination operator && could be replaced by || without any failure of T1.

4.4 Why Statement Coverage Alone Was Insufficient

- Statement coverage only proves that a line is executed, not that the decision on that line is correct. An "if" without an else can have 100% statement coverage with the false branch never tested.
- It does not test conditions, so logic errors (&& versus ||, > versus >=, wrong constant) pass unnoticed.
- It ignores the invisible else path, where missing-code defects hide.
- The proof is mutation analysis: the injected faults and the existing defect below all survive T1.

Fault (injected or existing)	**Detected by T1?**	**Test that kills it**
&& changed to ||	No (both give true)	T2: age 16, income 50,000 (expected reject)
Existing defect: income > 25000 (specification says >= 25000)	No (not detected)	T4: age 30, income 25,000 (expected approve)
age >= 18 changed to age > 18	No	T5: age 18, income 50,000 (expected approve)
Income condition removed	No	T3: age 30, income 10,000 (expected reject)

4.5 Improved White-Box Strategy

TC	**Age**	**Income**	**A (age >= 18)**	**B (income >= 25,000)**	**Expected**	**Purpose**
T1	30	50,000	T	T	Approve	Decision true; MC/DC base
T2	16	50,000	F	T	Reject	A independently decides the result
T3	30	10,000	T	F	Reject	B independently decides the result
T4	30	25,000	T	T	Approve	Boundary of income (detects the real defect)
T5	18	50,000	T	T	Approve	Boundary of age
T6	17	50,000	F	T	Reject	Age boundary - 1
T7	30	24,999	T	F	Reject	Income boundary - 1

- **Branch coverage:** T1 and T2 give both outcomes of the decision (100%).

- **Condition / MC/DC coverage:** T1, T2 and T3 show that each condition independently changes the outcome (n + 1 = 3 tests for two conditions).
- **Path coverage:** with k sequential decisions there are 2^k paths; for this module there are 2 paths, both covered. For larger modules, basis-path testing (V(G) tests) is used.
- **Boundary tests:** T4 to T7 catch off-by-one defects; T4 would have revealed the defect in the code.

4.6 Justification

The revised approach moves the criterion from "every line executed" to "every outcome and every condition verified". Each added test detects a fault that the original test missed, so error-detection power increases from 0 of 4 faults detected to 4 of 4 with only six more tests. Statement coverage is kept as a minimum entry criterion, with branch coverage as the target and MC/DC for safety- or money-critical conditions.

Question 5

A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

Answer:

5.1 Errors in the Partitioning Strategy

No.	Error	Explanation
E1	Classes are based on sign, not on validity	The requirement defines valid ages as 0 to 100; "negative" and "positive" do not correspond to accept and reject behavior
E2	Zero is not covered	0 is neither negative nor positive, yet it is a valid boundary value
E3	"Positive" mixes valid and invalid	5, 50 and 100 are valid, but 101, 150 and 99999 are invalid; all are in the same class
E4	Upper limit ignored	Nothing in the partitions identifies 100 as the maximum
E5	Non-numeric data missing	Letters, symbols, blank and decimals (25.5) are not partitioned
E6	No boundary value analysis	-1, 0, 1, 99, 100, 101 are not considered, so off-by-one defects remain undetected

5.2 Why the Strategy Fails to Reflect Valid and Invalid Limits

- A valid equivalence class must contain values that the system treats identically. Positive numbers are not treated identically: the system accepts 100 and rejects 101.
- The only test data possible from the two classes (for example -5 and 30) detect none of the upper-limit or zero-handling defects.
- A faulty implementation such as "if (age > 0 && age < 100)" (which wrongly rejects 0 and 100) would pass both tests.
- Root cause: the tester partitioned the number line mathematically instead of deriving partitions from the requirement and from the system's responses.

5.3 Corrected Partitions

Class	Type	Description	Example	Expected result
V1	Valid	0 <= age <= 100	35	Accepted
I1	Invalid	age < 0	-5	Rejected: age cannot be negative
I2	Invalid	age > 100	150	Rejected: age cannot exceed 100
I3	Invalid	Decimal values	25.5	Rejected: whole numbers only
I4	Invalid	Non-numeric / special characters	abc, #	Rejected: enter a number
I5	Invalid	Blank / null	(empty)	Rejected: age is required
I6	Invalid	Very large number	99999999999	Rejected; no overflow

5.4 Boundary Value Analysis

Boundary	Test values (min - 1, min, min + 1, max - 1, max, max + 1)	Expected
Lower (0)	-1, 0, 1	Reject, accept, accept
Upper (100)	99, 100, 101	Accept, accept, reject

5.5 Justification

- Partitions now follow the requirement: one valid class and six invalid classes, each producing a distinct, verifiable response.
- The six BVA values expose the most common implementation errors (> versus >=, < versus <=, off-by-one on 0 and 100).
- Invalid classes I3 to I6 test robustness against malformed input, which prevents crashes and injection of unexpected data.
- Total of 13 tests (6 boundary + 1 valid + 6 invalid classes) provide full partition and boundary coverage with no redundant tests, so the validation is considerably more robust than the 2 tests originally possible.

Question 6

A developer claims full Condition Coverage for a complex `if(A && B

Answer:

Note: *the question in the paper is cut off after "if(A && B". It is answered below for the statement `if (A && B)` (A and B are Boolean conditions), and extended to a three-condition decision `(A && B) || C`.*

6.1 The Developer's Claim (Assumed Test Design)

Test	A	B	Decision (A && B)
T1	True	False	False
T2	False	True	False

The developer argues that A took both values (True in T1, False in T2) and B took both values (False in T1, True in T2), hence "100% condition coverage".

6.2 Errors Identified

No.	Error	Explanation
E1	Decision never true	In both tests A && B is False, so the true branch of the if statement is never executed; decision (branch) coverage is only 50%. Condition coverage does not imply decision coverage
E2	Short-circuit evaluation ignored	In T2, A is False so B is not evaluated at all in most languages (C, C++, Java). B = True is therefore never actually exercised; condition coverage for B is really 50%
E3	No combination (True, True)	The only combination that can reveal an && versus || defect, or a wrong action in the true branch, is missing
E4	Independent effect of conditions not shown	Neither test proves that changing one condition alone changes the outcome (no MC/DC)
E5	Claim not supported by evidence	Coverage was claimed from the table of values, not from a coverage tool or an execution log

6.3 Root Causes

- Confusing **condition coverage** (each atomic condition True and False) with **decision coverage** (the whole expression True and False).
- Treating the conditions as independent of each other and of the language semantics, ignoring short-circuit behavior.

- Choosing test data to "tick" each condition value rather than analysing the truth table of the expression.
- Not using a coverage tool or review to verify the claim.

6.4 Corrections

Truth table and the required tests for A && B

Test	A	B	Result	Role
T1	T	T	True	Decision true; base for independence pairs
T2	T	F	False	With T1 shows B independently affects the result
T3	F	T	False	With T1 shows A independently affects the result (B is not evaluated, but the outcome is determined by A)
T4 (optional)	F	F	False	Needed only for multiple-condition coverage (all 2^n combinations)

Tests T1, T2 and T3 (n + 1 = 3) give 100% condition coverage, decision coverage and MC/DC. Adding T4 gives multiple-condition coverage (4 tests).

Extension: (A && B) || C

Test	A	B	C	Result	Independence shown
t1	T	T	F	True	Base
t2	F	T	F	False	A (t1 versus t2)
t3	T	F	F	False	B (t1 versus t3)
t4	F	T	T	True	C (t2 versus t4)

6.5 Justification

- The corrected test set executes both branches of the decision, so statement, branch and condition coverage are all achieved together.
- MC/DC proves that every condition can independently decide the outcome, which is the defect class (wrong operator, missing or negated condition) that the original tests could not reveal.
- Only n + 1 tests are needed instead of 2^n, so the approach scales to complex conditions in safety-critical software (for example DO-178C requires MC/DC).
- Using a coverage tool to confirm the result replaces an unsupported claim with evidence.

Question 7

A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

Answer:

7.1 Business Rules and the QA Team's Table (Assumed)

- A Premium Member gets a 10% discount.
- A valid coupon gives an additional 5% discount, but only if the order value is at least Rs. 1,000.
- Discounts add together (a premium member with a valid coupon on an eligible order gets 15%).

The team's table covered only three rules:

Rule	Premium member	Coupon applied	Discount
R1	Yes	No	10%
R2	No	Yes	5%
R3	No	No	0%

7.2 Errors in the Construction

No.	Error	Consequence
E1	Combination "Premium = Yes and Coupon = Yes" is missing	The stacking rule (10% + 5%) is untested; the system may apply only one discount or 15% of a wrong base
E2	Order-value condition (>= Rs. 1,000) not included	Coupon acceptance below Rs. 1,000 is untested
E3	Not all 2^n combinations enumerated	With 3 conditions there are 8 rules, not 3
E4	Coupon condition is vague ("applied")	Valid, expired and invalid coupons are not distinguished
E5	No expected result (price) in the table	Actions are percentages only; final prices are not calculated, so rounding errors are missed
E6	No completeness or redundancy check	Impossible and redundant rules were not identified; no review of the table

7.3 Why the Current Approach Is Incomplete

Decision table testing is only effective when the table is complete: n conditions require 2^n rules before any simplification. With 3 conditions the team tested 3 of 8 combinations (37.5%). The missing five rules include the highest-value case (premium plus coupon) and the failure cases, which is where price calculation errors occurred in production.

7.4 Revised Exhaustive Decision Table

Conditions: C1 = Premium member; C2 = Valid coupon entered; C3 = Order value >= Rs. 1,000.

Rule	C1	C2	C3	Action: discount	Example (order Rs. 2,000 / Rs. 800)
R1	Y	Y	Y	15%	Rs. 1,700
R2	Y	Y	N	10%; coupon rejected with message	Rs. 720 (order Rs. 800)
R3	Y	N	Y	10%	Rs. 1,800
R4	Y	N	N	10%	Rs. 720
R5	N	Y	Y	5%	Rs. 1,900
R6	N	Y	N	0%; coupon rejected with message	Rs. 800
R7	N	N	Y	0%	Rs. 2,000
R8	N	N	N	0%	Rs. 800

Simplified table: rules R3 and R4 differ only in C3 with the same result (when C2 = N, C3 does not matter), and so do R7 and R8. They can be merged using "-" (don't care), giving 6 rules; however, all 8 are kept as test cases because the cost is small.

Additional boundary and negative tests: order value Rs. 999.99 / 1,000 / 1,000.01 (C3 boundary); expired coupon; invalid coupon code; coupon used twice; rounding with prices such as Rs. 1,234.57 at 15%.

7.5 Justification

- All 8 combinations of the conditions are tested, so the stacking and eligibility rules are verified in every situation.
- Expected prices are calculated in the table, so a calculation error (wrong base, double discount, rounding) is detected, not only the percentage.
- Boundary tests on C3 detect "> 1000" versus ">= 1000" defects.

- The table is a reviewable specification: business analysts can confirm the rules before testing, so defects in requirements are also caught.
- The test suite can be automated directly from the table, and re-run for every change in discount policy.

Question 8

During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.

Answer:

8.1 Loop Under Test (Assumed)

A module averages up to 100 readings: `for (i = 0; i < n; i++) sum += a[i]; avg = sum / n;` where the array `a` has 100 elements (max = 100).

8.2 Errors in the Tester's Strategy

No.	Error	Defect that remains hidden
E1	Zero iterations (n = 0) not tested	Division by zero in sum / n; uninitialised average
E2	Over-maximum (n = max + 1 = 101) not tested	Array overflow (a[100] out of bounds), memory corruption
E3	Only first and last iteration considered	No test of n = 2 (loop-carried state errors) or of a typical value
E4	Max - 1 not tested	Off-by-one that stops the loop one iteration early or late
E5	Loop-control variable and exit condition not checked	"<" versus "<=" errors in the condition go unnoticed
E6	Nested and sequential loops not considered	Interactions between loop counters and shared variables untested

8.3 Root Causes

- **"Normal" thinking:** the tester assumed that if the loop works for the first and last iteration it works for all, ignoring that failures occur at the edges of the loop control.
- **Boundaries seen as valid values only:** n = 0 and n = max + 1 are invalid or special values, and the tester tested only the valid range.
- **No loop-testing checklist:** the standard loop test set (skip, once, twice, typical, max - 1, max, max + 1) was not used.
- **Specification gaps:** the behavior for an empty input or for more than max inputs was not defined, so it was not tested.

8.4 Corrected Approach for a Simple Loop

TC	Iterations (n)	Purpose	Expected result
1	0	Skip the loop	Defined handling, e.g. error or average 0; no division by zero
2	1	Single pass	avg = a[0]
3	2	Two passes: state carried from one iteration to the next	Correct average of two values
4	m = 50 (typical)	Normal operation	Correct average
5	max - 1 = 99	Just below limit	Correct average
6	max = 100	At the limit	Correct average
7	max + 1 = 101	Beyond the limit	Rejected or limited; no overflow

8.5 Nested Loops (Beizer's Approach)

1. Start with the innermost loop; set all outer loops to their minimum iteration values.
2. Test the innermost loop with the simple-loop tests (0, 1, 2, typical, max - 1, max, max + 1) while the outer loops stay at the minimum.
3. Move outward one loop at a time: outer loop at 0, 1, 2, typical, max - 1, max, max + 1 with the inner loops at typical values.
4. Finally test with all loops at their maximum, and at min/max combinations, to check interactions.
5. For concatenated loops that share a variable, treat them as nested; if independent, test them separately.

Example: a 3-level nest with 100 iterations per level would need 10^6 executions to cover all paths. The Beizer method reduces this to roughly 7 tests per loop level, about 7 + 7 + 7 + 1 = 22 tests plus the combined maximum case, which is practical and still hits every edge condition.

8.6 Justification (Path Integrity)

- Zero, one, two, typical and boundary iteration counts represent every distinct behavior of the loop: not entered, entered once, repeated, and at the limits. Each is a separate path through the loop in the control flow graph.
- n = 0 and n = max + 1 expose the two most dangerous defects (division by zero and buffer overflow).
- Tests at 2 and max - 1 catch faults in loop-carried state and in the exit condition.

- The nested-loop procedure avoids the exponential explosion of paths while preserving the boundary coverage, so path integrity is checked at a manageable cost.

Question 9

An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

Answer:

9.1 Errors in Relying Solely on Error Guessing

No.	Error	Why it is unacceptable for a braking system
E1	Test selection depends on the tester's experience and intuition	Different engineers guess different cases; unknown hazards remain untested
E2	No systematic coverage of inputs	Pedal pressure, speed and wheel-slip ranges are not partitioned, so limits are not verified
E3	Boundaries not tested deliberately	The ABS threshold (for example slip = 15%) and the emergency-brake threshold may be wrong by one unit
E4	System states and transitions ignored	Unsafe transitions (e.g. ABS active to idle without release) and fault handling are not explored
E5	No measurable coverage and no traceability to requirements or hazards	Compliance evidence (ISO 26262, ASIL-D) cannot be produced
E6	Not repeatable	Regression testing after a software change is not reliable
E7	Structured black-box methods deliberately ignored	Error guessing is a supplement, not a replacement

9.2 Impact on Test Adequacy

- **Coverage is unknown:** nobody can state which requirements, ranges or states were verified, so adequacy cannot be demonstrated.
- **Critical defects escape:** off-by-one errors at thresholds and rare state sequences cause failures only in real driving conditions, with severe consequences.
- **Inconsistent results:** the quality of testing varies with the tester, and the same release can pass or fail depending on who tested it.
- **Certification risk:** the safety case for the product is not supported by objective evidence.

9.3 Corrected Strategy: Boundary Value Analysis

Assumed parameters: pedal position 0 to 100%, activation of braking at pedal > 5%, emergency braking at pedal >= 90%, vehicle speed 0 to 200 km/h, ABS activation when wheel slip > 15%.

Parameter	Boundary	Test values	Expected behavior
Pedal position	Brake activation (5%)	4.9, 5.0, 5.1	No braking, no braking, normal braking
Pedal position	Emergency (90%)	89.9, 90, 90.1	Normal, emergency, emergency
Pedal position	Range limits	-0.1 (invalid), 0, 100, 100.1 (invalid)	Reject invalid values; handled safely
Wheel slip	ABS (15%)	14.9, 15.0, 15.1	No ABS, no ABS, ABS active
Speed	Range	0, 0.1, 199.9, 200, 200.1	Correct braking force; fault flag for out-of-range sensor

9.4 Corrected Strategy: State Transition Testing

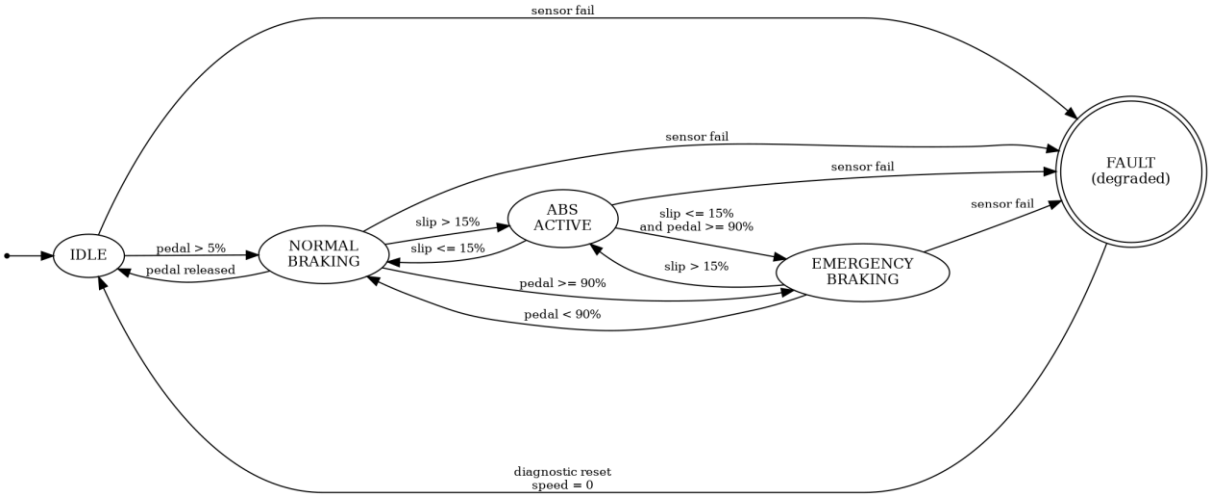


Figure 9.1: State transition diagram of the braking system (assumed)

The states are IDLE, NORMAL BRAKING, ABS ACTIVE, EMERGENCY BRAKING and FAULT. FAULT is a safe degraded state entered from any state upon a sensor failure and left only by a diagnostic reset at zero speed.

TC	Start state	Event	Expected end state	Type
S1	IDLE	Pedal > 5%	NORMAL	Valid
S2	NORMAL	Pedal released	IDLE	Valid
S3	NORMAL	Slip > 15%	ABS ACTIVE	Valid
S4	ABS ACTIVE	Slip <= 15%	NORMAL	Valid

TC	Start state	Event	Expected end state	Type
S5	NORMAL	Pedal >= 90%	EMERGENCY	Valid
S6	EMERGENCY	Slip > 15%	ABS ACTIVE	Valid
S7	Any state	Sensor failure	FAULT; warning lamp; degraded braking available	Valid
S8	FAULT	Diagnostic reset at speed 0	IDLE	Valid
N1	IDLE	Slip > 15% (no braking)	No transition (stay IDLE)	Invalid
N2	ABS ACTIVE	Pedal released	Defined safe handling; no direct jump to an undefined state	Invalid
N3	FAULT	Pedal > 5%	Stay in FAULT with degraded braking	Invalid
N4	FAULT	Reset while speed > 0	Reset rejected	Invalid

Sequences (1-switch coverage): IDLE-NORMAL-ABS-NORMAL-IDLE; IDLE-NORMAL-EMERGENCY-ABS-EMERGENCY; any state-FAULT-IDLE.

9.5 Justification of Increased Safety Assurance

- BVA verifies every threshold that controls a braking decision with the n - 1, n, n + 1 values, removing the off-by-one defects that guessing may miss.
- State transition testing covers every state, every valid transition and the dangerous invalid ones, including the behavior in FAULT, which is the safety-critical situation.
- Coverage is measurable (100% of boundaries, states and transitions) and traceable to safety requirements, as certification requires.
- Tests are repeatable and automatable (hardware-in-the-loop), so every software change can be fully re-verified.
- Error guessing is retained as a supplement (for example wet roads, sensor noise), applied after structured tests, so that expert intuition adds to rather than replaces coverage.

Question 10

A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.

Answer:

10.1 Code Under Test (Assumed)

Node	Code
1	read(user, amt, bal);
2	if (user is valid)
3	if (amt > 0)
4	if (amt <= bal)
5	{ bal = bal - amt; print "OK"; }
6	else print "Insufficient funds";
7	else print "Invalid amount";
8	else print "Invalid user";
9	end

10.2 The Tester's Incorrect CFG

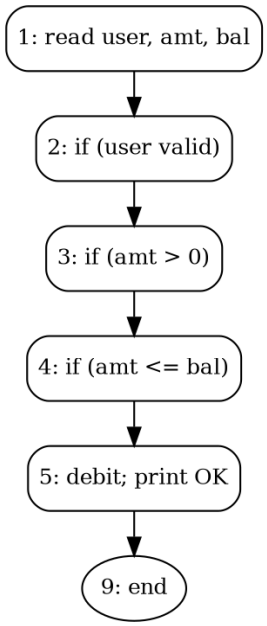


Figure 10.1: Incorrect CFG (only the success path is drawn)

10.3 Errors in Graph Construction

No.	Error	Effect
E1	Nodes 6, 7 and 8 (the else branches) are missing	Error messages are not part of the model
E2	Each decision node (2, 3, 4) has only the "true" outgoing edge	A decision with one outcome is not a decision; the false edges are not counted
E3	Join node 9 receives only one incoming edge	Merging of paths is lost
E4	Cyclomatic complexity not recomputed (graph shows V(G) = 1 instead of 4)	Only one path is believed to exist
E5	Coverage computed on the incomplete graph	The tool reports 100% because every node in the drawn graph is executed, while the omitted error statements and false edges are never counted or executed: branch coverage on the error paths is 0%

10.4 Root Causes of the Missing Nodes

- **Implicit else:** only the main (success) logic was modelled; else clauses written as one-line print statements, or not written at all, were overlooked.
- **Nested structure misread:** the tester did not map each "if" to a node with two outgoing edges, particularly when the else belongs to an outer if (the "dangling else").
- **Happy-path bias:** the test designer concentrated on the correct business flow and ignored validation and error handling.
- **No verification:** the graph was not cross-checked with the code or with the cyclomatic complexity formula.
- **Wrong coverage baseline:** coverage was measured against the drawn model instead of against the source code.

10.5 Corrected CFG

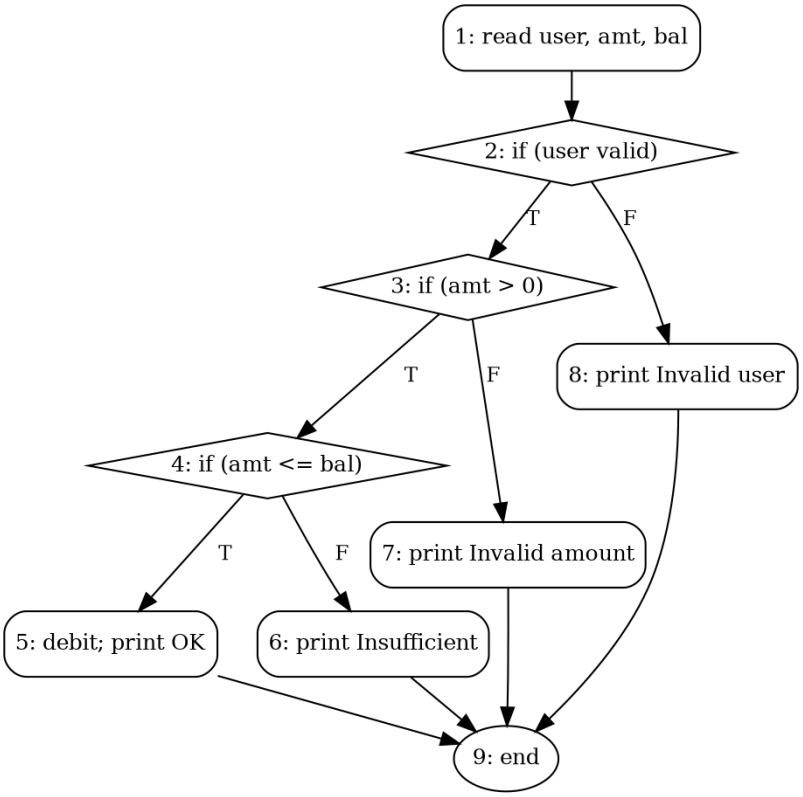


Figure 10.2: Corrected CFG with all else branches

Nodes N = 9, edges E = 11, so V(G) = E - N + 2 = 11 - 9 + 2 = **4**. Check: predicate nodes (2, 3, 4) + 1 = 4. The graph has 6 decision outcomes (3 decisions x 2) that must be covered for 100% branch coverage.

10.6 Independent Paths and Test Cases

Path	Node sequence	Test data (user, amt, bal)	Expected output	Branches covered
P1	1-2-3-4-5-9	valid, 500, 1,000	OK; balance 500	2T, 3T, 4T
P2	1-2-3-4-6-9	valid, 1,500, 1,000	Insufficient funds	4F
P3	1-2-3-7-9	valid, 0 (or -50), 1,000	Invalid amount	3F
P4	1-2-8-9	invalid, 500, 1,000	Invalid user	2F

Boundary additions: amt = 1 (valid minimum), amt = 0 (invalid), amt = bal (valid, balance becomes zero), amt = bal + 1 (insufficient).

10.7 Branch Adequacy

Measure	Incorrect CFG / tests	Corrected CFG / tests
Statement coverage (real code)	6 of 9 nodes = about 67% (nodes 6, 7, 8 not executed)	100%

Measure	Incorrect CFG / tests	Corrected CFG / tests
Branch coverage	3 of 6 outcomes = 50% overall; 0% of error branches	6 of 6 = 100%
Independent paths covered	1 of 4 = 25%	4 of 4 = 100%
Tests needed	1	4 (plus 4 boundary tests)

- Every decision now has both a true and a false edge, and each false edge leads to a node representing the error handling, so the model is faithful to the code.
- The four tests execute every error path, which is where defects in validation and messages are most likely.
- Calculating V(G) and verifying it against the number of tests makes branch adequacy provable.
- Coverage should always be measured on the source code with a tool, not on a hand-drawn model.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand